

Course overview

C++ is built for **control, speed, and flexibility**. Unlike many high-level languages, it lets you manage memory, define abstractions, use templates, and write code close to hardware while still supporting modern programming techniques.

A strong C++ course should cover:

- Syntax and compilation.
- Functions and modular design.
- Pointers and references.
- Classes and objects.
- Constructors and destructors.
- STL and algorithms.
- Smart pointers and RAII.
- Templates and generic programming.
- Move semantics.
- Concurrency.
- Modern C++20 features.

What C++ is

C++ is an extension and evolution of C that adds object-oriented programming, generic programming, and modern language features. It is widely used when performance and control are important, and advanced courses often focus on both the language itself and the broader techniques it teaches.

A useful way to think about it is:

- C gives you low-level control.
- C++ gives you low-level control plus better abstraction tools.
- Modern C++ gives you safer memory management and expressive syntax.

How to implement C++

To start implementing C++, you need a compiler, an editor, and a build workflow. The usual setup is:

1. Install a compiler such as GCC, Clang, or MSVC.
2. Use an editor or IDE such as VS Code or Visual Studio.
3. Write .cpp files.

BrainStack

4. Compile them into an executable.
5. Run and debug the output.

Basic compile command:

bash

```
g++ main.cpp -o app
```

```
./app
```

A minimal C++ program:

```
cpp
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    cout << "Hello, C++!" << endl;
```

```
    return 0;
```

```
}
```

Core syntax

Every advanced C++ learner should master these basics first:

- Variables and data types.
- Input and output.
- Conditionals.
- Loops.
- Functions.
- Arrays and strings.
- Pointers and references.

BrainStack

Example:

```
cpp
#include <iostream>

using namespace std;

int add(int a, int b) {
    return a + b;
}

int main() {
    int x = 5, y = 7;
    cout << "Sum = " << add(x, y) << endl;
    return 0;
}
```

Object-oriented programming

C++ becomes much more powerful when you use classes and objects. OOP lets you group data and behavior together, which is useful for building scalable software.

Example:

```
cpp
#include <iostream>

using namespace std;

class Student {
private:
    string name;
    int age;
public:
    Student(string n, int a) {
        name = n;
        age = a;
    }
}
```

BrainStack

```
void display() {  
    cout << name << " " << age << endl;  
}  
};  
  
int main() {  
    Student s1("Aman", 21);  
    s1.display();  
    return 0;  
}
```

Advanced concepts

The advanced parts of C++ are what make it a serious university-level subject. These include:

- RAII.
- Smart pointers.
- Move semantics.
- Templates.
- Template metaprogramming.
- Lambdas.
- STL algorithms.
- Multithreading.
- Atomic operations.
- Exception safety.
- C++17 and C++20 features.

RAII and smart pointers

RAII means that resources are acquired and released through object lifetime, which makes code safer. Smart pointers like `unique_ptr`, `shared_ptr`, and `weak_ptr` help manage memory automatically.

Move semantics

Move semantics improve performance by transferring ownership instead of copying heavy objects. This is a major modern C++ feature.

BrainStack

Templates

Templates let you write generic code that works with many types. This is one of the biggest strengths of C++.

Multithreading

C++ supports concurrency through `std::thread`, `std::mutex`, `std::future`, and atomic types. This is important for performance and real-world systems.

Basic projects with code

Here are some projects that match a university-style C++ course.

1. Calculator

cpp

```
#include <iostream>

using namespace std;

int main() {
    double a, b;
    char op;
    cin >> a >> op >> b;
    if (op == '+') cout << a + b;
    else if (op == '-') cout << a - b;
    else if (op == '*') cout << a * b;
    else if (op == '/') cout << a / b;
    else cout << "Invalid operator";
    return 0;
}
```

2. Bank account system

cpp

```
#include <iostream>

using namespace std;

class BankAccount {
    string owner;
    double balance;
```

public:

```
BankAccount(string name, double bal) : owner(name), balance(bal) {}  
  
void deposit(double amount) { balance += amount; }  
  
void withdraw(double amount) { if (amount <= balance) balance -= amount; }  
  
void show() {  
    cout << owner << " balance: " << balance << endl;  
}
```

```
};
```

```
int main() {  
    BankAccount acc("Ravi", 1000);  
    acc.deposit(500);  
    acc.withdraw(300);  
    acc.show();  
    return 0;  
}
```

3. Student management

cpp

```
#include <iostream>  
  
#include <vector>  
  
using namespace std;  
  
struct Student {  
    string name;  
    int marks;  
};  
  
int main() {  
    vector<Student> students = {{"Asha", 90}, {"Rahul", 85}};  
    for (auto &s : students) {  
        cout << s.name << " " << s.marks << endl;  
    }  
}
```

BrainStack

```
    return 0;
}
```

4. Number guessing game

cpp

```
#include <iostream>

using namespace std;

int main() {
    int secret = 7, guess;

    cout << "Guess the number: ";

    cin >> guess;

    if (guess == secret) cout << "Correct!";

    else cout << "Try again!";

    return 0;
}
```

5. To-do list

cpp

```
#include <iostream>

#include <vector>

using namespace std;

int main() {
    vector<string> tasks;

    tasks.push_back("Learn C++");
    tasks.push_back("Build project");

    for (string task : tasks) {
        cout << task << endl;
    }

    return 0;
}
```

Best project ideas

Good advanced C++ projects include:

- Banking management system.
- Student record system.
- Library management system.
- File encryption tool.
- Console-based chat application.
- Snake game.
- Sudoku solver.
- Task manager.
- Inventory system.
- Multithreaded downloader.

How to study C++ effectively

The best way to learn C++ is to:

1. Read the concept.
2. Write the code by hand.
3. Compile and debug.
4. Improve the design.
5. Turn each concept into a mini-project.

C++ becomes much easier when you connect syntax with real projects instead of memorizing theory alone.